

Task 15: Service Registry and Discovery using Eureka

AIM:

To implement Service Registry and Service Discovery using Eureka by registering multiple microservices and enabling dynamic service lookup instead of using hard-coded service URLs.

PROCEDURE:

1. Create a Eureka Server (Service Registry).
2. Add required dependencies for Eureka Server.
3. Enable Eureka Server using annotation.
4. Create multiple microservices (clients).
5. Add Eureka Client dependency in each microservice.
6. Configure application properties to register services with Eureka Server.
7. Enable service registration using annotations.
8. Start Eureka Server.
9. Run all microservices.
10. Verify that all services are registered in Eureka Dashboard.
11. Use service names instead of hard-coded URLs for communication.

ALGORITHM:

1. Start
2. Create Eureka Server
3. Add `@EnableEurekaServer` annotation
4. Run Eureka Server on default port (8761)
5. Create Microservice 1 and Microservice 2
6. Add Eureka Client dependency
7. Configure application.properties with Eureka Server URL
8. Enable client using `@EnableDiscoveryClient`
9. Run all microservices
10. Register services to Eureka Server
11. Fetch services dynamically using service names

12. Stop

PROGRAM:

1. Eureka Server Application

@SpringBootApplication

@EnableEurekaServer

```
public class EurekaServerApplication {  
    public static void main(String[] args) {  
        SpringApplication.run(EurekaServerApplication.class, args);  
    }  
}  
}
```

application.properties (Eureka Server)

server.port=8761

eureka.client.register-with-eureka=false

eureka.client.fetch-registry=false

2. Microservice 1 (Client)

@SpringBootApplication

@EnableDiscoveryClient

```
public class Service1Application {  
    public static void main(String[] args) {  
        SpringApplication.run(Service1Application.class, args);  
    }  
}
```

application.properties (Service 1)

spring.application.name=service-1

server.port=8081

eureka.client.service-url.defaultZone=http://localhost:8761/eureka/

3. Microservice 2 (Client)

@SpringBootApplication

@EnableDiscoveryClient

```
public class Service2Application {  
    public static void main(String[] args) {  
        SpringApplication.run(Service2Application.class, args);  
    }  
}
```

application.properties (Service 2)

spring.application.name=service-2

server.port=8082

eureka.client.service-url.defaultZone=http://localhost:8761/eureka/

4. Service Communication using RestTemplate

@Bean

@LoadBalanced

```
public RestTemplate restTemplate() {  
    return new RestTemplate();  
}
```

@Autowired

private RestTemplate restTemplate;

```
public String callService() {  
    return restTemplate.getForObject("http://service-2/endpoint", String.class);  
}
```

OUTPUT:

spring Eureka

HOME LAST 1000 SINCE STARTUP

System Status

Environment	test	Current time	2024-05-26T11:30:15 +0530
Data center	default	Uptime	00:06
		Lease expiration enabled	false
		Renews threshold	3
		Renews (last min)	5

DS Replicas

Instances currently registered with Eureka

Application	AMIs	Availability Zones	Status
SERVICE-1	n/a (1)	(1)	UP (1) - DESKTOP-1:service-1:8081
SERVICE-2	n/a (1)	(1)	UP (1) - DESKTOP-1:service-2:8082

2. Run Service 1 (port 8081)

```
2024-05-26 11:28:15.123 INFO 12340 --- [main] c.e.s1.Service1Application : Starting Service1Application
2024-05-26 11:28:16.789 INFO 12340 --- [main] o.s.c.n.e.s.EurekaServiceRegistry : Registering application SERVICE-1
2024-05-26 11:28:16.912 INFO 12340 --- [main] o.s.c.n.e.s.EurekaServiceRegistry : Registered Instance SERVICE-1:8081
2024-05-26 11:28:16.958 INFO 12340 --- [main] c.e.s1.Service1Application : Started Service1Application in 2.345 seconds
```

3. Run Service 2 (port 8082)

```
2024-05-26 11:28:20.234 INFO 5678 --- [main] c.e.s2.Service2Application : Starting Service2Application
2024-05-26 11:28:21.456 INFO 5678 --- [main] o.s.c.n.e.s.EurekaServiceRegistry : Registering application SERVICE-2
2024-05-26 11:28:21.587 INFO 5678 --- [main] o.s.c.n.e.s.EurekaServiceRegistry : Registered Instance SERVICE-2:8082
2024-05-26 11:28:21.600 INFO 5678 --- [main] c.e.s2.Service2Application : Started Service2Application in 2.101 seconds
```

4. Call Service 1 endpoint which calls Service 2 using service name

← → ↻ ⓘ http://localhost:8081/call-service2

Response from Service 2: Hello from Service 2!

What we observe:

- Eureka server shows both services registered.
- Services are UP.
- Service 1 successfully calls Service 2 using service name ([http://service-2/...](http://service-2/)) instead of hard-coded URL.

RESULT:

The Service Registry and Discovery system was successfully implemented using Eureka.

- Microservices were registered dynamically.
- Hard-coded service URLs were eliminated.
- Service discovery enabled scalable and flexible microservice communication.